# Bài tập thực hành tuần 3

> Họ và tên: Nguyễn Vạn Phúc Huy
> MSSV: 23110163
> Lớp: 23TTH (Sáng thứ 6)

In [17]:
```
import matplotlib.pyplot as plt
import numpy as np
import time
import random
import sys
from time import perf_counter
from math import log2


# Từ Python 3.11+, giới hạn số chữ số cho các phép chuyển đổi int <-> str bị giới hạn ở 4300.
# Cần cấu hình nới lỏng giới hạn này để chạy được N lớn (ví dụ N = 2^12 = 4096).
sys.set_int_max_str_digits(1000000)
sys.setrecursionlimit(10000)
```

## Bài tập 1:

**Câu hỏi**:

**Bài 1.** Viết một hàm trong đó:

- Input: $f(n)$, hai số nguyên dương $a, b$ chứa khoảng giá trị của $n$. Ví dụ $a = 10, b = 1000$. Khi đó $n$ chạy từ $10 \to 1000$.
- Output: Chỉ ra độ phức tạp của $f(n) = O(n^\alpha)$, nếu không có dạng này hiện thông báo.

Kiểm tra lại hàm đã có với các trường hợp sau:

a) $f(n) = n^2$.

b) $f(n) = n^3 + \cos(n).n^4$.

c) $f(n) = n^n$.

d) $f(n) = n^3 + n^2 + n + 1$.

Note: Output của bài 1 yêu cầu là CHỈ RA độ phức tạp của f(n) = O(n^a), nếu không thì hiện thông báo. Vì vậy, ta có thể chỉ cần làm thuật toán để kiểm tra điều đó.

In [18]:
```
def findN_Alpha(func):
    arr = np.linspace(10, 1000)
    vals = np.abs(func(arr))

    alphas = np.log(vals) / np.log(arr)
    max_alpha = np.nanmax(alphas)

    if max_alpha == np.inf:
```

```
        print(f' O(n^a) not exist')
        return None
    else:
        print(f' f ~= O(n^{round(max_alpha)}) ')
        return max_alpha

def func_a(n):
    return n**2
def func_b(n):
    return n**3 + np.cos(n) * n**4
def func_c(n):
    return n**n
def func_d(n):
    return n**3 + n**2 + n + 1

print("a)", end="")
findN_Alpha(func_a)
print("b)", end="")
findN_Alpha(func_b)
print("c)", end="")
findN_Alpha(func_c)
print("d)", end="")
findN_Alpha(func_d)
```

```
a) f ~= O(n^2)
b) f ~= O(n^4)
c) O(n^a) not exist
d) f ~= O(n^3)
```

```
C:\Users\ADMIN\AppData\Local\Temp\ipykernel_16544\4176933670.py:20: RuntimeWarning: overflow encountered in power
  return n**n
```

Out[18]:
```
np.float64(3.0457140589408676)
```

```
def findN_Alpha(func):
    arr = np.linspace(10, 1000)
    vals = np.abs(func(arr))

    alphas = np.log(vals) / np.log(arr)
    max_alpha = np.nanmax(alphas)

    if max_alpha == np.inf:
        print(f' O(n^a) not exist')
        return None
    else:
        print(f' f ~= O(n^{round(max_alpha)}) ')
        return max_alpha
```

Đầu tiên, ta tạo một mảng `arr` từ 10 đến 1000 để kiểm tra giá trị của hàm `func` tại các điểm này. Sau đó, ta tính giá trị tuyệt đối của `func(arr)` và lưu vào `vals`.

```
arr = np.linspace(10, 1000)
vals = np.abs(func(arr))
```

Tiếp theo, ta tính giá trị của `alpha` bằng cách lấy logarit của `vals` chia cho logarit của `arr` theo như trong phần hướng dẫn. Điều này giúp ta xác định độ phức tạp của hàm `func`.

```
alphas = np.log(vals) / np.log(arr)
```

Sau đó, ta tìm giá trị lớn nhất của `alpha` bằng cách sử dụng `np.nanmax()`, vì có thể có giá trị NaN nếu `vals` chứa giá trị âm hoặc bằng 0.

```
max_alpha = np.nanmax(alphas)
```

Cuối cùng, ta kiểm tra nếu `max_alpha` là vô hạn (`np.inf`), điều đó có nghĩa là hàm `func` không có độ phức tạp O(n^a) nào tồn tại. Nếu không, ta in ra độ phức tạp gần đúng của hàm `func`.

```
if max_alpha == np.inf:
    print(f' O(n^a) not exist')
    return None
else:
    print(f' f ~= O(n^{round(max_alpha)}) ')
    return max_alpha
```

Kết quả thu được là:

Câu a: f(n) = O(n^2)

Câu b: f(n) = O(n^4)

Câu c: Không tìm được

Câu d: f(n) = O(n^3)

Nhưng nếu ta muốn tìm các hệ số a0, a1, a2,... thì ta có thể sử dụng phương pháp hồi quy tuyến tính để tìm các hệ số này. Dưới đây là một ví dụ về cách thực hiện điều đó:

In [19]:
```
import math
import numpy as np

def estimate_complexity(f, a, b):
    n_vals = []
    fn_vals = []
    log_n = []
    log_fn = []

    # BƯỚC 1: Lấy dữ liệu và chuyển sang dạng Logarit
    for n in range(a, b + 1):

            val = f(n)
            abs_val = abs(val) # Xét trị tuyệt đối cho độ lớn của hàm

            if abs_val > 0:
                # Xử lý các số cực lớn (vd n^n) để tránh lỗi tràn bộ nhớ (Overflow)
                if isinstance(abs_val, int) and abs_val > 1e250:
                    l_fn = abs_val.bit_length() * math.log(2)
                else:
                    l_fn = math.log(abs_val)

                n_vals.append(n)
                fn_vals.append(val)
                log_n.append(math.log(n))
                log_fn.append(l_fn)


    if len(n_vals) < 2:
        print("giá trị n quá nhỏ hoặc hàm trả về giá trị không hợp lệ để ước lượng độ phức tạp.")
        return

    # Lọc đường bao trên (Upper Envelope) để tìm xu hướng thật sự
    # Kỹ thuật này giúp loại bỏ nhiễu khi hàm dao động lên xuống do cos(n)
    env_log_n, env_log_fn = [], []
    max_val = -float('inf')
    for ln, lfn in zip(log_n, log_fn):
        if lfn >= max_val:
            max_val = lfn
            env_log_n.append(ln)
            env_log_fn.append(lfn)

    # Chia đôi dữ liệu bao trên để kiểm tra xem hệ số góc có ổn định không
    # Nếu hệ số góc tăng vọt -> Hàm tăng phi đa thức (như hàm mũ n^n)
    mid = len(env_log_n) // 2
    if mid >= 2:
        slope1 = np.polyfit(env_log_n[:mid], env_log_fn[:mid], 1)[0]
        slope2 = np.polyfit(env_log_n[mid:], env_log_fn[mid:], 1)[0]
        if slope2 - slope1 > 0.5:
            print("Ko có dạng O(n^a)")
            return

    # Tính độ dốc trung bình của toàn bộ đường bao trên
    overall_slope = np.polyfit(env_log_n, env_log_fn, 1)[0]
    alpha = int(round(overall_slope))

    if alpha == 0:
        print("f(n) ~= O(1)")
    elif alpha == 1:
        print("f(n) ~= O(n)")
    else:
        print(f"f(n) ~= O(n^{alpha})")
```

```python
    # BƯỚC 2: Thế f(n) và n để giải hệ phương trình tìm a_0, a_1,..., a_alpha
        # np.polyfit(X, Y, deg) trả về hệ số đa thức theo thứ tự: [a_alpha, ..., a_1, a_0]
        coeffs = np.polyfit(n_vals, fn_vals, alpha)

        # Đảo ngược mảng để in đúng thứ tự yêu cầu của đề: a_0, a_1, ..., a_alpha
        coeffs_reversed = coeffs[::-1]

        # Làm tròn hệ số để hiển thị sạch sẽ (khử sai số dấu phẩy động 0.0000001)
        coeffs_rounded = [round(c, 4) for c in coeffs_reversed]

        print(" => Các hệ số [a_0, a_1, ..., a_alpha]:")
        str_coeffs = [f"a_{i} = {c}" for i, c in enumerate(coeffs_rounded)]
        print("    " + ", ".join(str_coeffs))

        # In ra dạng đa thức trực quan
        poly_terms = []
        for i, c in enumerate(coeffs_rounded):
            if c != 0:
                if i == 0: poly_terms.append(f"{c}")
                elif i == 1: poly_terms.append(f"{c}n")
                else: poly_terms.append(f"{c}n^{i}")

        poly_str = " + ".join(poly_terms).replace("+ -", "- ")
        print(f" => Hàm xấp xỉ: f(n) ≈ {poly_str}")

def func_a(n): return n**2
def func_b(n): return n**3 + math.cos(n) * (n**4)
def func_c(n): return n**n
def func_d(n): return n**3 + n**2 + n + 1

a, b = 10, 1000
print("a) f(n) = n^2")
estimate_complexity(func_a, a, b)

print("b) f(n) = n^3 + cos(n).n^4")
estimate_complexity(func_b, a, b)

print("c) f(n) = n^n")
estimate_complexity(func_c, a, b)

print("d) f(n) = n^3 + n^2 + n + 1")
estimate_complexity(func_d, a, b)
```

```
a) f(n) = n^2
f(n) ~= O(n^2)
 => Các hệ số [a_0, a_1, ..., a_alpha]:
    a_0 = -0.0, a_1 = 0.0, a_2 = 1.0
 => Hàm xấp xỉ: f(n) ≈ 1.0n^2
b) f(n) = n^3 + cos(n).n^4
f(n) ~= O(n^4)
 => Các hệ số [a_0, a_1, ..., a_alpha]:
    a_0 = 6530563598.812, a_1 = -140495447.621, a_2 = 709460.8461, a_3 = -1235.1968, a_4 = 0.6868
 => Hàm xấp xỉ: f(n) ≈ 6530563598.812 - 140495447.621n + 709460.8461n^2 - 1235.1968n^3 + 0.6868n^4
c) f(n) = n^n
Ko có dạng O(n^a)
d) f(n) = n^3 + n^2 + n + 1
f(n) ~= O(n^3)
 => Các hệ số [a_0, a_1, ..., a_alpha]:
    a_0 = 1.0, a_1 = 1.0, a_2 = 1.0, a_3 = 1.0
 => Hàm xấp xỉ: f(n) ≈ 1.0 + 1.0n + 1.0n^2 + 1.0n^3
```

## Bài tập 2:

**Câu hỏi**:

**Bài 2.** Viết chương trình nhân 2 số nguyên lớn $A, B$ có $N$ chữ số.

- Input: $A, B$ và $N$.
- Output: $C = A.B$.

Bằng 2 phương pháp:

a) Phương pháp truyền thống có độ phức tạp $O(N^2)$.

b) Phương pháp cải tiến $O(N^{\log 3})$.

Kiểm tra lại chương trình với điều kiện sau:

- Lấy $N = 2^k, k = 10, 11, \ldots, 32$. $A, B$ là 2 số nguyên có $N$ chữ số lấy ngẫu nhiên. Ứng với mỗi giá trị của $N$, tính thời gian trung bình để $A.B$.



- So sánh 2 phương pháp để xác định hai phương pháp có độ phức tạp là $O(N^2)$ và $O(N^{\log 3})$.

In [20]:
```
# Bài 2
def listRNG(N):
    n = []
    for _ in range(N):
        n.append(random.randint(0, 9))

    while n[0] == 0:
        n[0] = random.randint(1, 9)
    return n

def to_list(n):
    l = []
    while n:
        l.insert(0, n % 10)
        n //= 10

    return l

def list_to_number(digits):
    number = 0
    for digit in digits:
        number = number * 10 + digit
    return number

def plot_time_complexity(T, bigO_line, bigO_label, title=''):

    plt.figure(figsize=(8, 5))
    ax = plt.gca()
    ax2 = ax.twinx()
```

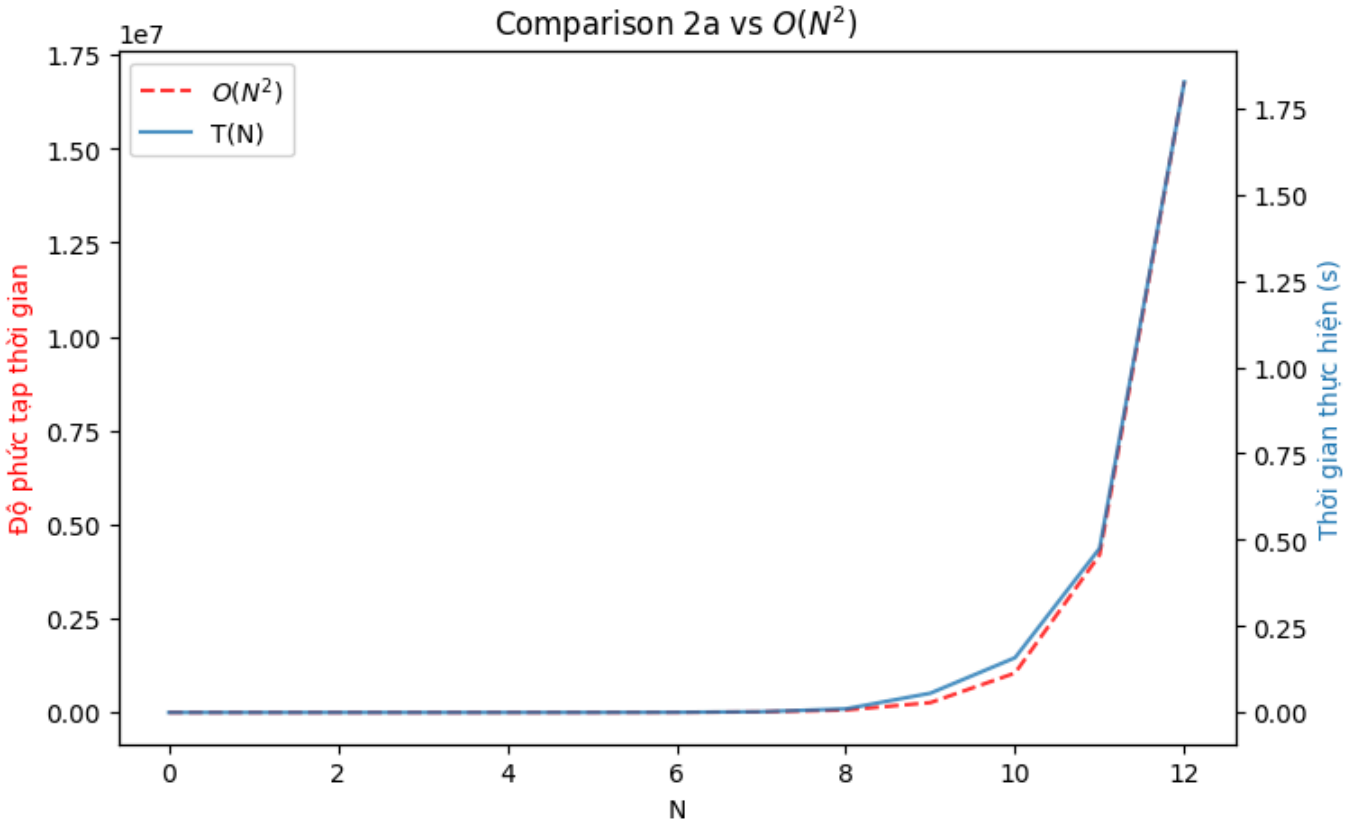
```python
    ax.set_xlabel('N')

    line1, = ax.plot(bigO_line, label=bigO_label, color='r', ls='--', alpha=0.8)
    ax.set_ylabel('Độ phức tạp thời gian', color=line1.get_color())
    line2, = ax2.plot(T, label='T(N)', color='C0', alpha=0.8)
    ax2.set_ylabel('Thời gian thực hiện (s)', color=line2.get_color())
    ax.legend(handles=[line1, line2])

    if title:
        plt.title(title)

    plt.show()

# 2a
def classic_multiply(A, B):
    len_a, len_b = len(A), len(B)
    res = [0] * (len_a + len_b)

    for i in range(len_a - 1, -1, -1):
        carry = 0
        for j in range(len_b - 1, -1, -1):
            temp = A[i] * B[j] + res[i + j + 1] + carry
            res[i + j + 1] = temp % 10
            carry = temp // 10
        res[i] += carry

    while len(res) > 1 and res[0] == 0:
        res.pop(0)

    return int("".join(map(str, res)))


T_a = []
Ns_a = []
for k in range(13):
    N = 2 ** k
    Ns_a.append(N)
    A = listRNG(N)
    B = listRNG(N)

    tic = perf_counter()
    classic_multiply(A, B)
    toc = perf_counter()

    duration = toc - tic
    print(f'{k = }, took {duration}s')
    T_a.append(duration)



bigO_line_a = list(map(lambda x: x**2, Ns_a))
plot_time_complexity(T_a, bigO_line_a, bigO_label='$O(N^2)$',title=f'Comparison 2a vs $O(N^2)$')

# 2b
def split(n: int):
    return divmod(n, 10**(numlength(n)//2))

def numlength(n: int):
    l = 0
    while n:
        n //= 10
        l += 1
    return l

def karatsuba_multiply(A, B):
    # Điều kiện dừng đệ quy
    if A < 10 or B < 10:
        return int(A * B)

    # Tìm mốc chia đôi m dựa trên số dài nhất
    str_A, str_B = str(A), str(B)
    n = max(len(str_A), len(str_B))
    m = n // 2
```

```python
    # Lũy thừa để chia cắt số
    power = 10 ** m

    # Chia A = A_1 * 10^m + A_2
    A_1, A_2 = divmod(A, power)
    B_1, B_2 = divmod(B, power)

    # 3 lệnh gọi đệ quy Karatsuba
    C = karatsuba_multiply(A_1, B_1)          # Tương đương z2
    D = karatsuba_multiply(A_2, B_2)          # Tương đương z0
    E = karatsuba_multiply(A_1 + A_2, B_1 + B_2) - C - D  # Tương đương z1 - z2 - z0

    # Gộp lại theo mốc m đã định: C * 10^(2m) + E * 10^m + D
    return (C * (10 ** (2 * m))) + (E * power) + D

def rng(N):
    n = random.randint(0, 9)
    while n == 0:
        n = random.randint(0, 9)

    i = 1
    while i < N:
        n = n*10 + random.randint(0, 9)
        i += 1
    return n


T_b = []
Ns_b = []
for k in range(13):
    N = 2 ** k
    Ns_b.append(N)
    A = rng(N)
    B = rng(N)

    tic = perf_counter()
    karatsuba_multiply(A, B)
    toc = perf_counter()

    duration = toc - tic
    print(f'{k = }, took {duration}s')
    T_b.append(duration)

bigO_line_b = list(map(lambda x: x**log2(3) , Ns_b))
plot_time_complexity(T_b, bigO_line_b, bigO_label='$O(N^{\log{3}})$',title='Comparison 2b vs $O(N^{\log{3}}$



# So sánh 2 phương pháp
# Tính hệ số C để chuẩn hóa đường lý thuyết về cùng đơn vị thời gian (giây)
C_a = T_a[-1] / bigO_line_a[-1]
C_b = T_b[-1] / bigO_line_b[-1]

bigO_line_a_scaled = [x * C_a for x in bigO_line_a]
bigO_line_b_scaled = [x * C_b for x in bigO_line_b]

plt.figure(figsize=(8, 5))
plt.plot(Ns_a, bigO_line_a_scaled, label='$O(N^2)$', color='red', ls='--', alpha=0.5)
plt.plot(Ns_a, T_a, label='$T_a$ (Classical)', color='red', marker='o')

plt.plot(Ns_a, bigO_line_b_scaled, label='$O(N^{\log_2{3}})$', color='blue', ls='--', alpha=0.5)
plt.plot(Ns_a, T_b, label='$T_b$ (Karatsuba)', color='blue', marker='s')

plt.xlabel('N')
plt.ylabel('s')
plt.legend()
plt.title('Comparison of Classical and Karatsuba Multiplication (Normalized)')
plt.grid(True, alpha=0.3)
plt.show()
```

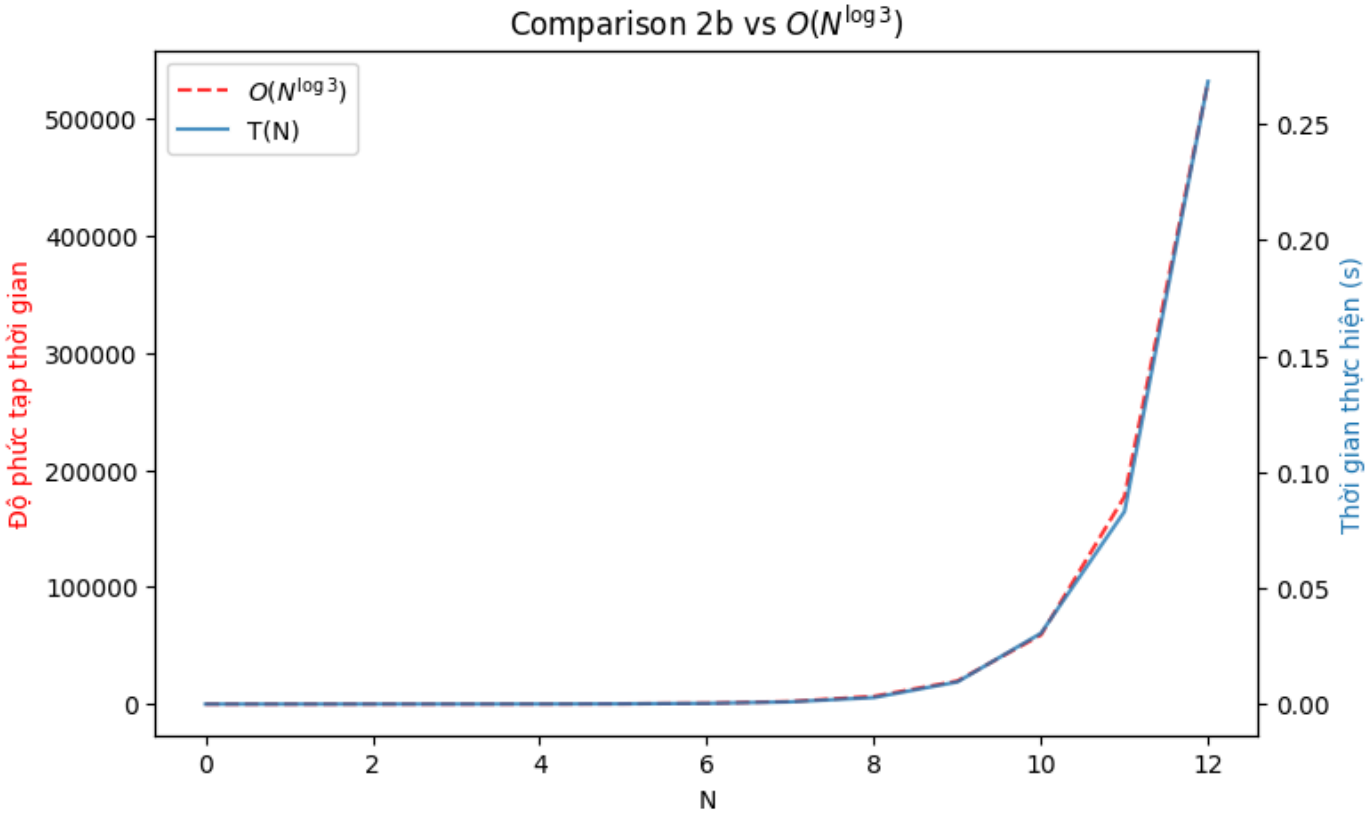
```
k = 0, took 1.3499999113264494e-05s
k = 1, took 1.0599998859106563e-05s
k = 2, took 8.299999535665847e-06s
k = 3, took 1.559999873279594e-05s
k = 4, took 5.419999979494605e-05s
k = 5, took 0.00014379999993252568s
k = 6, took 0.0005185000009078067s
k = 7, took 0.002188399999795365s
k = 8, took 0.010462700000061886s
k = 9, took 0.05579989999932877s
```

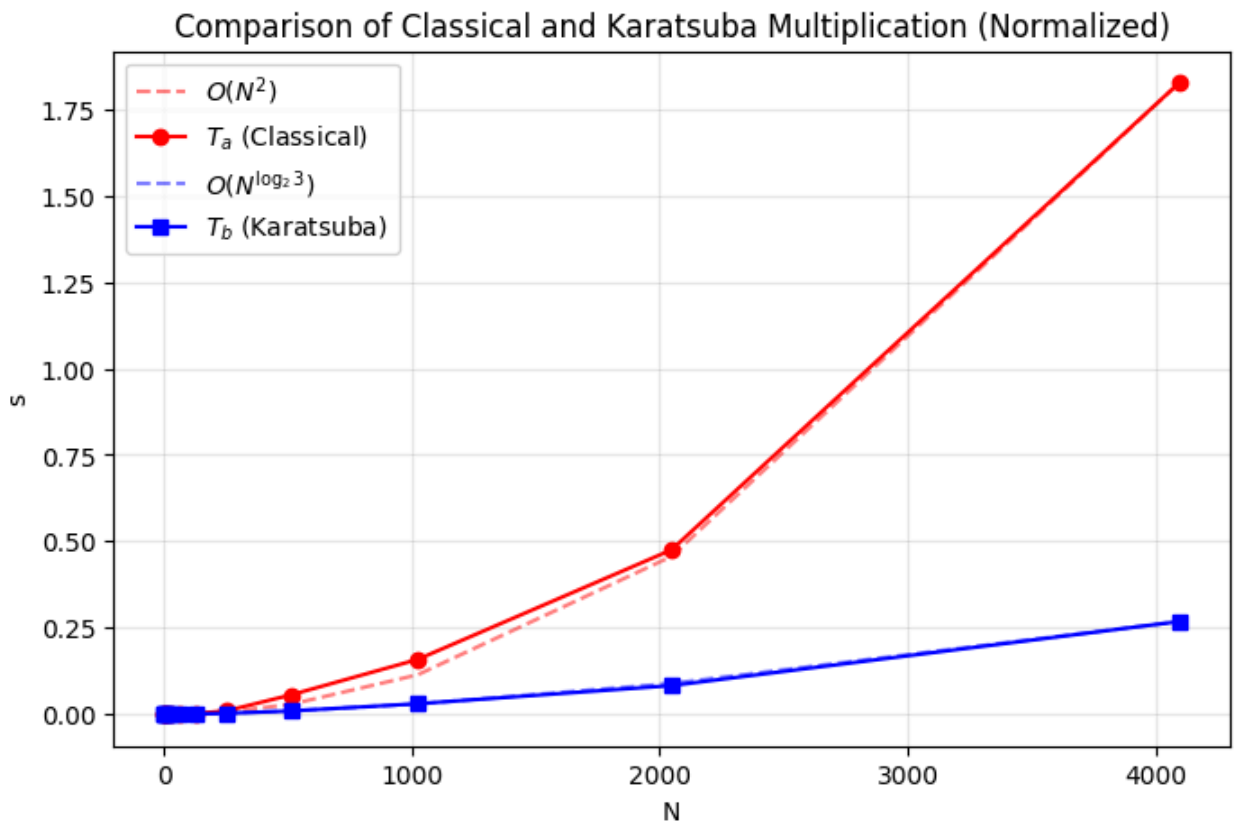
```
<>:148: SyntaxWarning: invalid escape sequence '\l'
<>:148: SyntaxWarning: invalid escape sequence '\l'
<>:164: SyntaxWarning: invalid escape sequence '\l'
<>:148: SyntaxWarning: invalid escape sequence '\l'
<>:148: SyntaxWarning: invalid escape sequence '\l'
<>:164: SyntaxWarning: invalid escape sequence '\l'
C:\Users\ADMIN\AppData\Local\Temp\ipykernel_16544\3367751692.py:148: SyntaxWarning: invalid escape sequence
'\l'
  plot_time_complexity(T_b, bigO_line_b, bigO_label='$O(N^{\log{3}})$',title='Comparison 2b vs $O(N^{\log
{3}})$')
C:\Users\ADMIN\AppData\Local\Temp\ipykernel_16544\3367751692.py:148: SyntaxWarning: invalid escape sequence
'\l'
  plot_time_complexity(T_b, bigO_line_b, bigO_label='$O(N^{\log{3}})$',title='Comparison 2b vs $O(N^{\log
{3}})$')
C:\Users\ADMIN\AppData\Local\Temp\ipykernel_16544\3367751692.py:164: SyntaxWarning: invalid escape sequence
'\l'
  plt.plot(Ns_a, bigO_line_b_scaled, label='$O(N^{\log_2{3}})$', color='blue', ls='--', alpha=0.5)
```

```
k = 10, took 0.15851269999984652s
k = 11, took 0.47546929999953136s
k = 12, took 1.8260451999994984s
```




```
k = 0, took 1.1000010999850929e-06s
k = 1, took 4.300000000512227e-06s
k = 2, took 7.700000423938036e-06s
k = 3, took 1.630000042496249e-05s
k = 4, took 4.3199999709031545e-05s
k = 5, took 0.00011619999895629007s
k = 6, took 0.0002982000005431473s
k = 7, took 0.0009589000001142267s
k = 8, took 0.002816500000335509s
k = 9, took 0.009543299998767907s
k = 10, took 0.030528399998729583s
k = 11, took 0.08298400000057882s
k = 12, took 0.2682532999988325s
```




Các hàm phụ trợ cho bài 2

```
def listRNG(N):
    n = []
    for _ in range(N):
        n.append(random.randint(0, 9))

    while n[0] == 0:
        n[0] = random.randint(1, 9)
    return n
```

Hàm `listRNG(N)` tạo ra một danh sách gồm N chữ số ngẫu nhiên từ 0 đến 9, đảm bảo rằng chữ số đầu tiên không phải là 0 để tránh trường hợp số có giá trị bằng 0.

```
def to_list(n):
    l = []
```

```
    while n:
        l.insert(0, n % 10)
        n //= 10

    return l
```

Hàm `to_list(n)` chuyển một số nguyên `n` thành một danh sách các chữ số của nó. Nó sử dụng phép chia lấy phần dư để tách từng chữ số và chèn chúng vào đầu danh sách.

```
def list_to_number(digits):
    number = 0
    for digit in digits:
        number = number * 10 + digit
    return number
```

Hàm `list_to_number(digits)` chuyển một danh sách các chữ số thành một số nguyên. Nó duyệt qua từng chữ số trong danh sách và xây dựng số bằng cách nhân số hiện tại với 10 và cộng thêm chữ số tiếp theo.

```
def numlength(n: int):
    l = 0
    while n:
        n //= 10
        l += 1
    return l
```

Hàm `numlength(n)` tính độ dài của một số nguyên `n` bằng cách liên tục chia `n` cho 10 và đếm số lần chia cho đến khi `n` trở thành 0. Kết quả trả về là số lượng chữ số của `n`.

```
def split(n: int):
    return divmod(n, 10**(numlength(n)//2))
```

Hàm `split(n)` chia một số nguyên `n` thành hai phần bằng cách sử dụng hàm `numlength` để xác định độ dài của số và sau đó sử dụng `divmod` để tách số thành phần cao và phần thấp dựa trên mốc chia đôi. Kết quả trả về là một tuple chứa hai phần của số.

```
def rng(N):
    n = random.randint(0, 9)
    while n == 0:
        n = random.randint(0, 9)

    i = 1
    while i < N:
        n = n*10 + random.randint(0, 9)
        i += 1
    return n
```

Hàm `rng(N)` tạo ra một số nguyên ngẫu nhiên có N chữ số. Nó đảm bảo rằng chữ số đầu tiên không phải là 0 bằng cách kiểm tra và sinh lại nếu cần. Sau đó, nó xây dựng số bằng cách nhân số hiện tại với 10 và cộng thêm một chữ số ngẫu nhiên mới cho đến khi đạt được N chữ số.

```
def plot_time_complexity(T, bigO_line, bigO_label, title=''):

    plt.figure(figsize=(8, 5))
    ax = plt.gca()
    ax2 = ax.twinx()
    ax.set_xlabel('N')

    line1, = ax.plot(bigO_line, label=bigO_label, color='r', ls='--', alpha=0.8)
    ax.set_ylabel('Độ phức tạp thời gian', color=line1.get_color())
    line2, = ax2.plot(T, label='T(N)', color='C0', alpha=0.8)
    ax2.set_ylabel('Thời gian thực hiện (s)', color=line2.get_color())
    ax.legend(handles=[line1, line2])

    if title:
        plt.title(title)

    plt.show()
```

Hàm `plot_time_complexity(T, bigO_line, bigO_label, title='')` tạo ra một biểu đồ so sánh giữa thời gian thực tế `T` và đường lý thuyết `bigO_line`. Nó sử dụng hai trục y để hiển thị cả độ phức tạp thời gian và thời gian thực hiện. Hàm này cũng cho phép tùy chỉnh tiêu đề và nhãn của các đường trên biểu đồ.

**Câu 2a:**

```
def classic_multiply(A, B):
    len_a, len_b = len(A), len(B)
    res = [0] * (len_a + len_b)

    for i in range(len_a - 1, -1, -1):
        carry = 0
        for j in range(len_b - 1, -1, -1):
            temp = A[i] * B[j] + res[i + j + 1] + carry
            res[i + j + 1] = temp % 10
            carry = temp // 10
        res[i] += carry

    while len(res) > 1 and res[0] == 0:
        res.pop(0)

    return int("".join(map(str, res)))
```

Hàm `classic_multiply(A, B)` thực hiện phép nhân hai số nguyên được biểu diễn dưới dạng danh sách các chữ số. Nó sử dụng phương pháp nhân truyền thống, tương tự như cách chúng ta nhân tay, bằng cách nhân từng chữ số của hai số và cộng dồn kết quả vào một danh sách `res` để lưu trữ kết quả trung gian. Cuối cùng, nó loại bỏ các chữ số 0 ở đầu và chuyển đổi danh sách kết quả thành một số nguyên.

```
    len_a, len_b = len(A), len(B)
    res = [0] * (len_a + len_b)
```

Đầu tiên, hàm xác định độ dài của hai số A và B, sau đó tạo một danh sách `res` có kích thước đủ lớn để chứa kết quả của phép nhân (tối đa là tổng độ dài của hai số).

```
    for i in range(len_a - 1, -1, -1):
        carry = 0
        for j in range(len_b - 1, -1, -1):
            temp = A[i] * B[j] + res[i + j + 1] + carry
            res[i + j + 1] = temp % 10
            carry = temp // 10
        res[i] += carry
```

Hàm sử dụng hai vòng lặp để nhân từng chữ số của A với từng chữ số của B. Kết quả tạm thời được lưu vào `temp`, sau đó cập nhật danh sách `res` và phần nhớ (carry) cho các phép nhân tiếp theo.

```
    while len(res) > 1 and res[0] == 0:
        res.pop(0)
```

Sau khi hoàn thành phép nhân, hàm loại bỏ các chữ số 0 ở đầu của kết quả để đảm bảo rằng số trả về không có các chữ số 0 thừa.

```
    return int("".join(map(str, res)))
```

Cuối cùng, hàm chuyển đổi danh sách `res` thành một chuỗi và sau đó thành một số nguyên để trả về kết quả cuối cùng của phép nhân.

Trong câu 2a, ta đã đo thời gian thực hiện của hàm `classic_multiply` cho các giá trị N khác nhau và lưu kết quả vào danh sách `T_a`. Sau đó, ta tạo một đường lý thuyết O(N^2) và sử dụng hàm `plot_time_complexity` để so sánh thời gian thực tế với đường lý thuyết.

```
T_a = []
Ns_a = []
for k in range(13):
    N = 2 ** k
    Ns_a.append(N)
    A = listRNG(N)
    B = listRNG(N)

    tic = perf_counter()
    classic_multiply(A, B)
    toc = perf_counter()

    duration = toc - tic
    print(f'{k = }, took {duration}s')
    T_a.append(duration)
```

```
bigO_line_a = list(map(lambda x: x**2, Ns_a))
plot_time_complexity(T_a, bigO_line_a, bigO_label='$O(N^2)$',title=f'So sánh thuật toán câu 2a với $O(N^2)$')
```

**Câu 2b:** Hàm `karatsuba_multiply(A, B)` thực hiện phép nhân hai số nguyên sử dụng thuật toán Karatsuba, một thuật toán đệ quy có độ phức tạp tốt hơn so với phương pháp nhân truyền thống. Thuật toán này chia mỗi số thành hai phần và thực hiện ba phép nhân đệ quy để tính toán kết quả cuối cùng.

```
def karatsuba_multiply(A, B):
    if A < 10 or B < 10:
        return int(A * B)

    str_A, str_B = str(A), str(B)
    n = max(len(str_A), len(str_B))
    m = n // 2

    power = 10 ** m

    A_1, A_2 = divmod(A, power)
    B_1, B_2 = divmod(B, power)

    C = karatsuba_multiply(A_1, B_1)
    D = karatsuba_multiply(A_2, B_2)
    E = karatsuba_multiply(A_1 + A_2, B_1 + B_2) - C - D

    return (C * (10 ** (2 * m))) + (E * power) + D
```

Trong câu 2b, ta đã đo thời gian thực hiện của hàm `karatsuba_multiply` cho các giá trị N khác nhau và lưu kết quả vào danh sách `T_b`. Sau đó, ta tạo một đường lý thuyết O(N^log2(3)) và sử dụng hàm `plot_time_complexity` để so sánh thời gian thực tế với đường lý thuyết.

```
T_b = []
Ns_b = []
for k in range(13):
    N = 2 ** k
    Ns_b.append(N)
    A = rng(N)
    B = rng(N)

    tic = perf_counter()
    karatsuba_multiply(A, B)
    toc = perf_counter()

    duration = toc - tic
    print(f'{k = }, took {duration}s')
    T_b.append(duration)
bigO_line_b = list(map(lambda x: x**log2(3) , Ns_b))
plot_time_complexity(T_b, bigO_line_b, bigO_label='$O(N^{\log{3}})$',title='Comparison 2b vs $O(N^{\log{3}})$')
```

Cuối cùng, ta đã chuẩn hóa cả hai đường lý thuyết về cùng đơn vị thời gian (giây) bằng cách tính hệ số C dựa trên giá trị cuối cùng của T_a và T_b so với giá trị cuối cùng của bigO_line_a và bigO_line_b. Sau đó, ta đã vẽ biểu đồ so sánh giữa thời gian thực tế của cả hai thuật toán với đường lý thuyết đã chuẩn hóa.

```
C_a = T_a[-1] / bigO_line_a[-1]
C_b = T_b[-1] / bigO_line_b[-1]
bigO_line_a_scaled = [x * C_a for x in bigO_line_a]
bigO_line_b_scaled = [x * C_b for x in bigO_line_b]
plt.figure(figsize=(8, 5))
plt.plot(Ns_a, bigO_line_a_scaled, label='$O(N^2)$', color='red', ls='--', alpha=0.5)
plt.plot(Ns_a, T_a, label='$T_a$ (Classical)', color='red', marker='o')
plt.plot(Ns_a, bigO_line_b_scaled, label='$O(N^{\log_2{3}})$', color='blue', ls='--',
alpha=0.5)
plt.plot(Ns_a, T_b, label='$T_b$ (Karatsuba)', color='blue', marker='s')
plt.xlabel('N')
plt.ylabel('s')
plt.title('Comparison of Classical and Karatsuba Multiplication')
```

```
plt.legend()
plt.show()
```